

0

DATA BASE SYSTEMS ARCHITECTURE

Databases III

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Lecturer
Department of Computer Engineer
School of Engineering
Universidad Distrital Francisco José de Caldas

2025-I



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

- 1 Database System Administration
- 2 Record Storage
- 3 DBMS Architecture
- 4 Transactional System
- 5 Query Execution
- 6 Concurrency Control
- 7 Failure Recovery



Outline

- 1 Database System Administration
- 2 Record Storage
- 3 DBMS Architecture
- 4 Transactional System
- 5 Query Execution
- 6 Concurrency Control
- 7 Failure Recovery



Database System Administration

- **Database system administration** is the discipline of **managing**, **configuring**, and **maintaining database systems** to ensure their **reliability**, **performance**, **security**, and **availability**.

- Key responsibilities include:

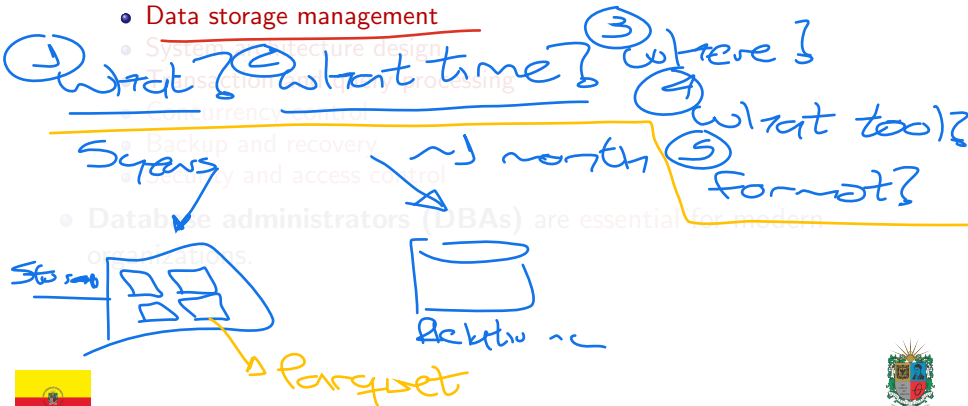


- Database administrators (DBAs) are essential for modern organizations.



Database System Administration

- **Database system administration** is the discipline of **managing**, **configuring**, and **maintaining** database systems to ensure their reliability, performance, security, and availability.
- Key responsibilities include:
 - Data storage management



Database System Administration

- **Database system administration** is the discipline of **managing**, **configuring**, and **maintaining** database systems to ensure their reliability, performance, security, and availability.
- Key responsibilities include:
 - Data storage management
 - System architecture design

physical
infrastructure

on-premise? cloud? hybrid?

logical architecture → SQL? NoSQL?
 m. nosql? sync? scaling?
 etl-elt? data lake?
 data warehouse? data mart?



Database System Administration

- **Database system administration** is the discipline of **managing**, **configuring**, and **maintaining** *database systems* to ensure their reliability, performance, security, and availability.

- Key responsibilities include:

- Data storage management
- System architecture design
- Transaction and query processing
- Concurrency control

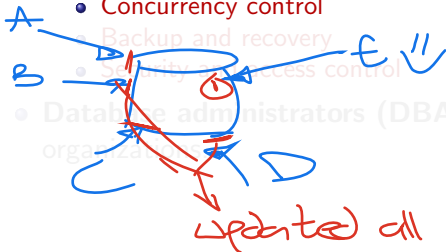
tool → failure
!!
☹

response time? concurrency?
order in query? optimization?



Database System Administration

- **Database system administration** is the discipline of **managing**, **configuring**, and **maintaining** *database systems* to ensure their reliability, performance, security, and availability.
- Key responsibilities include:
 - Data storage management
 - System architecture design
 - Transaction and query processing
 - Concurrency control



Database System Administration

- **Database system administration** is the discipline of **managing**, **configuring**, and **maintaining** *database systems* to ensure their reliability, performance, security, and availability.
- Key responsibilities include:

- Data storage management
- System architecture design
- Transaction and query processing
- Concurrency control
- Backup and recovery
- Security and access control

Database administrators (DBAs) are essential for modern organizations.

authentic recreation



fault tolerant?

copy-availability

self-recovery?



Database System Administration

- **Database system administration** is the discipline of **managing**, **configuring**, and **maintaining** *database systems* to ensure their reliability, performance, security, and availability.
- Key responsibilities include:
 - Data storage management
 - System architecture design
 - Transaction and query processing
 - Concurrency control
 - Backup and recovery
 - Security and access control

Database administrators (DBAs) are essential for modern applications.

SSH → Grants? → 2FA → auth 2 authenticator

↓ protocols



Database System Administration

- **Database system administration** is the discipline of **managing**, **configuring**, and **maintaining** *database systems* to ensure their reliability, performance, security, and availability.
- Key responsibilities include:
 - Data storage management
 - System architecture design
 - Transaction and query processing
 - Concurrency control
 - Backup and recovery
 - Security and access control
- **Database administrators (DBAs)** are **essential** for modern organizations.



Outline

- 1 Database System Administration
- 2 Record Storage**
- 3 DBMS Architecture
- 4 Transactional System
- 5 Query Execution
- 6 Concurrency Control
- 7 Failure Recovery



Record Storage Concepts

- A **record** (or row/tuple) is the basic unit of data storage in a database table.

- Efficient record storage is critical for fast data retrieval and update.
- Storage techniques:
 - Heap storage: fast insertion but slow retrieval.
 - B-tree storage: fast retrieval but slow insertion.



Record Storage Concepts

- A **record** (or row/tuple) is the **basic unit** of data storage in a *database table*.
- **Efficient record storage** is crucial for **fast data retrieval** and **update**.

• Storage techniques:

SELECT
(self)

1. Operation with
most executions

2. Cost $\rightarrow$ money
 $\rightarrow$ space

Create
Update
Delete
Save (row)



Record Storage Concepts

- A **record** (or row/tuple) is the **basic unit** of data storage in a *database table*.
- **Efficient record storage** is crucial for **fast data retrieval** and **update**.
- Storage techniques:

• Heap storage: Unordered, fast inserts.

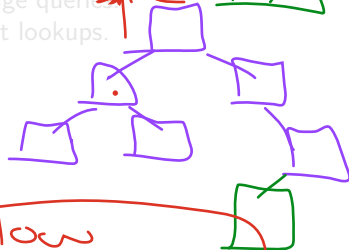
• Sequential storage: Ordered, fast range queries.

• Indexed storage: Uses indexes for fast lookups.



Insert &

Free



Slow Search

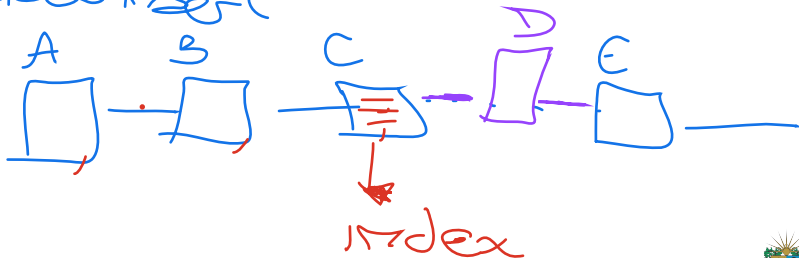


Record Storage Concepts

- A **record** (or row/tuple) is the **basic unit** of data storage in a *database table*.
- **Efficient record storage** is crucial for **fast data retrieval** and **update**.
- Storage techniques:
 - **Heap storage**: Unordered, fast inserts.
 - **Sequential storage**: Ordered, fast range queries.

Indexed storage: Uses indexes for fast lookups.

slow insert



Record Storage Concepts

- A **record** (or row/tuple) is the **basic unit** of data storage in a *database table*.
- **Efficient record storage** is crucial for **fast data retrieval** and **update**.
- Storage techniques:
 - **Heap storage**: Unordered, fast inserts.
 - **Sequential storage**: Ordered, fast range queries.
 - **Indexed storage**: Uses indexes for fast lookups.

Altra skua updates

A { 1 - ~~25~~ 22 → 12

B { ~~26~~ - ~~23~~ 27 - 35

C { ~~3~~ - ~~35~~ ~~45~~ 34 - 46



Block and Page Organization

- Data is stored in **blocks** (pages) on disk.

- Block size and layout affect I/O performance.

- Records may be packed, split or span multiple blocks.

- Free space management is important for updates and inserts.

- Fragmentation management ensures efficient use of space.

- Data compression can also improve storage efficiency.



video



Block and Page Organization

- Data is stored in **blocks** (pages) on disk.
- **Block size** and **layout** affect **I/O performance**.

- Records may be packed, slotted, or span multiple blocks.

- Free space management is important for updates and inserts.

- Fragmentation management ensures efficient use of space.

- Data compression can also improve storage efficiency.



Block and Page Organization

- Data is stored in **blocks** (pages) on disk.
- **Block size** and **layout** affect **I/O performance**.
- **Records** may be packed, slotted, or span multiple blocks.
- Free space management is important for updates and inserts.
- Fragmentation management ensures efficient use of space.
- Data compression can also improve storage efficiency.

+ performance

+ storage

+ hard to migrate



Block and Page Organization

- Data is stored in **blocks** (pages) on disk.
- **Block size** and **layout** affect **I/O performance**.
- Records may be **packed**, **slotted**, or **span multiple blocks**.
- **Free space management** is important for **updates** and **inserts**.

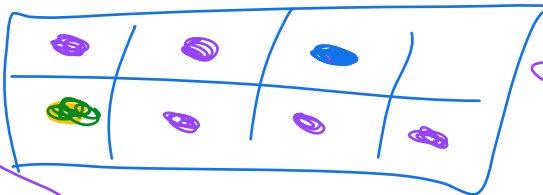
- Fragmentation management ensures efficient use of space.
- Data compression can also improve storage efficiency.

2GB → J.S.E.
Free



Block and Page Organization

- Data is stored in **blocks** (pages) on disk.
- **Block size** and **layout** affect **I/O performance**.
- Records may be **packed**, **slotted**, or **span multiple blocks**.
- **Free space management** is important for **updates** and **inserts**.
- **Fragmentation management** ensures efficient use of space.
- Data compression can also improve storage efficiency.



defragment



Block and Page Organization

- Data is stored in **blocks** (pages) on disk.
- **Block size** and **layout** affect **I/O performance**.
- Records may be **packed**, **slotted**, or **span multiple blocks**.
- **Free space management** is important for **updates** and **inserts**.
- **Fragmentation management** ensures efficient use of space.
- **Data compression** can also improve storage efficiency.

|
- redundant
+ Fast load

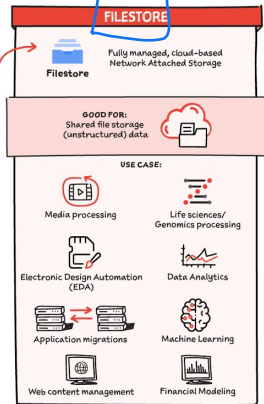
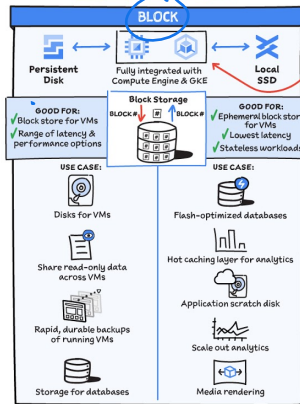
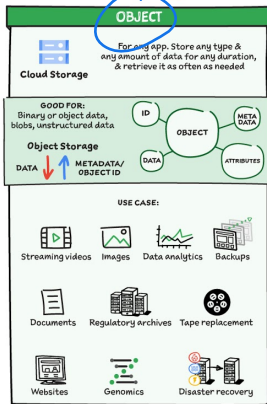


Record Storage: Image

#GCPsketchnote
 @PVERGADIA
 THECLOUDGIRL.DEV
 04.23.2021



Which Storage Should I Use?



Outline

- 1 Database System Administration
- 2 Record Storage
- 3 DBMS Architecture**
- 4 Transactional System
- 5 Query Execution
- 6 Concurrency Control
- 7 Failure Recovery



DBMS Architecture Overview

- Engines*
- A **Database Management System (DBMS)** is organized in layers:
 - Storage Manager**: Handles data storage, file organization, and access methods.

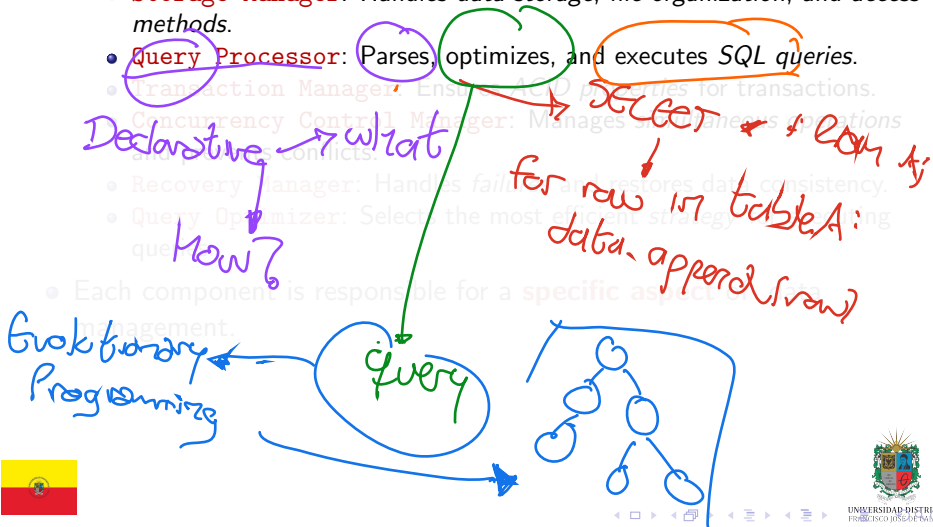


- Each component is responsible for a **specific aspect** of data management.



DBMS Architecture Overview

- A **Database Management System (DBMS)** is organized in **layers**:
 - **Storage Manager**: Handles *data storage*, file organization, and access methods.
 - **Query Processor**: Parses, optimizes, and executes SQL queries.
 - **Transaction Manager**: Ensures ACID properties for transactions.
 - **Concurrency Control Manager**: Manages simultaneous operations and prevents conflicts.
 - **Recovery Manager**: Handles failures and restores data consistency.
 - **Query Optimizer**: Selects the most efficient strategy for executing queries.
- Each component is responsible for a **specific aspect** of DBMS management.



DBMS Architecture Overview

- A **Database Management System (DBMS)** is organized in **layers**:

- **Storage Manager**: Handles *data storage*, *file organization*, and *access methods*.
- **Query Processor**: Parses, optimizes, and executes *SQL queries*.
- **Transaction Manager**: Ensures *ACID properties* for transactions.

- **Concurrency Control Manager**: Manages *simultaneous operations* and prevents conflicts.

- **Recovery Manager**: Handles *failures* and *restores data consistency*.

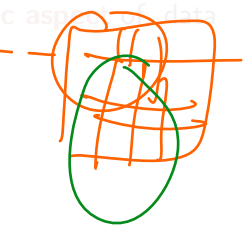
- **Query Optimizer**: Selects the most efficient strategy.

Atomicity → Debug, understanding
Consistency

Isolated

Durability

Queue request → execute



DBMS Architecture Overview

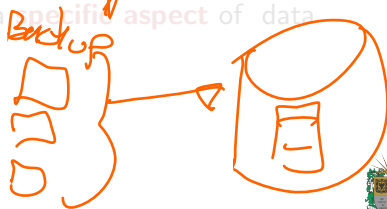
- A **Database Management System (DBMS)** is organized in **layers**:
 - **Storage Manager**: Handles *data storage*, *file organization*, and *access methods*.
 - **Query Processor**: Parses, optimizes, and executes *SQL queries*.
 - **Transaction Manager**: Ensures *ACID properties* for transactions.
 - **Concurrency Control Manager**: Manages *simultaneous operations* and prevents conflicts.



DBMS Architecture Overview

- A **Database Management System (DBMS)** is organized in **layers**:
 - **Storage Manager**: Handles *data storage*, *file organization*, and *access methods*.
 - **Query Processor**: Parses, optimizes, and executes *SQL queries*.
 - **Transaction Manager**: Ensures *ACID properties* for transactions.
 - **Concurrency Control Manager**: Manages *simultaneous operations* and prevents conflicts.
 - **Recovery Manager**: Handles *failures* and restores data consistency.
 - **Query Optimizer**: Selects the most efficient strategy for executing queries.

if I ~ bbbkey &
 then restart
 com A → B
 or crypt



DBMS Architecture Overview

- A **Database Management System (DBMS)** is organized in **layers**:
 - **Storage Manager**: Handles *data storage*, *file organization*, and *access methods*.
 - **Query Processor**: Parses, optimizes, and executes *SQL queries*.
 - **Transaction Manager**: Ensures *ACID properties* for transactions.
 - **Concurrency Control Manager**: Manages *simultaneous operations* and prevents conflicts.
 - **Recovery Manager**: Handles *failures* and restores data consistency.
 - **Query Optimizer**: Selects the most efficient *strategy* for executing queries.
- Each component is responsible for a **specific aspect** of data management.

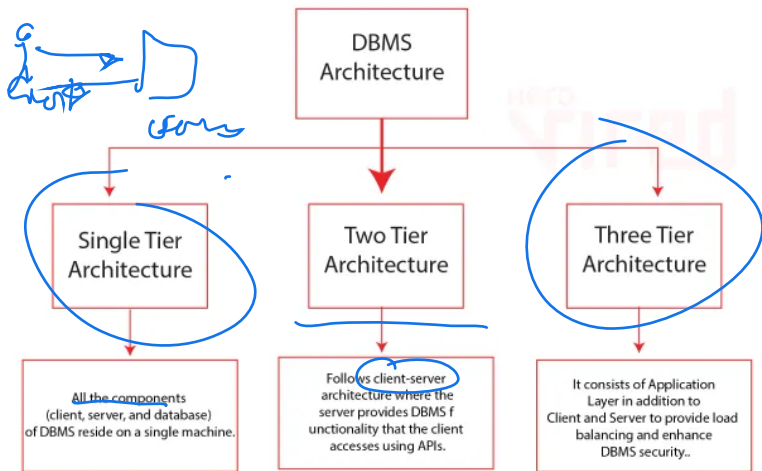


DBMS Architecture Overview

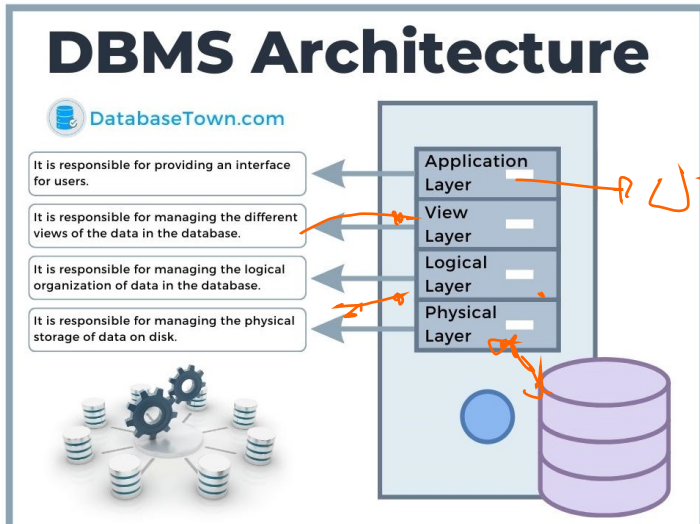
- A **Database Management System (DBMS)** is organized in **layers**:
 - **Storage Manager**: Handles *data storage*, *file organization*, and *access methods*.
 - **Query Processor**: Parses, optimizes, and executes *SQL queries*.
 - **Transaction Manager**: Ensures *ACID properties* for transactions.
 - **Concurrency Control Manager**: Manages *simultaneous operations* and prevents conflicts.
 - **Recovery Manager**: Handles *failures* and restores data consistency.
 - **Query Optimizer**: Selects the most efficient *strategy* for executing queries.
- Each component is responsible for a **specific aspect** of data management.



DBMS Architecture Tiers



DBMS Architecture N-Tier



Types of DBMS Architecture

There are several types of DBMS architectures:

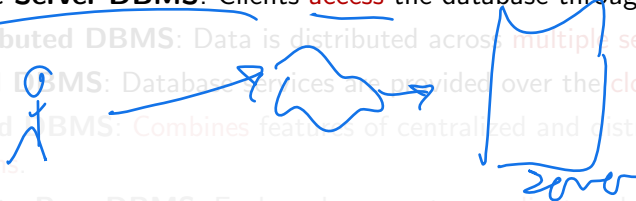
- Centralized DBMS: All components are on a **single server**.
- Client-Server DBMS: Clients **access** the database through a **server**.
- Distributed DBMS: Data is distributed across **multiple servers**.
- Cloud DBMS: Database services are provided over the **cloud**.
- Hybrid DBMS: **Combines** features of centralized and distributed systems.
- Peer-to-Peer DBMS: Each node can act as a **client** and **server**.
- In memory DBMS: Data is stored in **RAM** for faster access.



Types of DBMS Architecture

There are several types of **DBMS architectures**:

- **Centralized DBMS**: All components are on a **single server**.
- **Client-Server DBMS**: Clients **access** the database through a **server**.
- **Distributed DBMS**: Data is distributed across **multiple servers**.
- **Cloud DBMS**: Database services are provided over the **cloud**.
- **Hybrid DBMS**: Combines features of centralized and distributed systems.
- **Peer-to-Peer DBMS**: Each node can act as a **client** and **server**.
- **In memory DBMS**: Data is stored in **RAM** for faster access.



Types of DBMS Architecture

There are several types of **DBMS architectures**:

- **Centralized DBMS**: All components are on a **single server**.
- **Client-Server DBMS**: Clients **access** the database through a **server**.
- **Distributed DBMS**: Data is distributed across **multiple servers**.



Types of DBMS Architecture

There are several types of **DBMS architectures**:

- **Centralized DBMS**: All components are on a **single server**.
- **Client-Server DBMS**: Clients **access** the database through a **server**.
- **Distributed DBMS**: Data is distributed across **multiple servers**.
- **Cloud DBMS**: Database services are ~~provided over the cloud~~.
- **Hybrid DBMS**: Combines features of centralized and distributed systems. *\$ downside*
- **Peer-to-Peer DBMS**: Each node can act as a **client** and **server**.
- **In memory DBMS**: Data is stored in **RAM** for faster access.



Types of DBMS Architecture

There are several types of **DBMS architectures**:

- **Centralized DBMS**: All components are on a **single server**.
- **Client-Server DBMS**: Clients **access** the database through a **server**.
- **Distributed DBMS**: Data is distributed across **multiple servers**.
- **Cloud DBMS**: Database services are provided over the **cloud**.
- **Hybrid DBMS**: Combines features of centralized and distributed systems.
- **Peer-to-Peer DBMS**: Each node can act as a **client** and **server**.
- **In memory DBMS**: Data is stored in **RAM** for faster access.

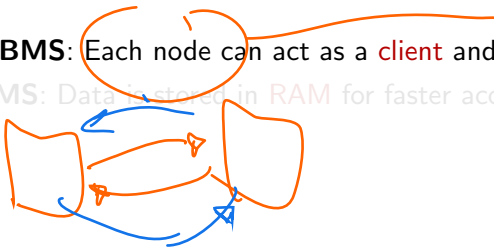
→ no grado
→ no tfg



Types of DBMS Architecture

There are several types of **DBMS architectures**:

- **Centralized DBMS**: All components are on a **single server**.
- **Client-Server DBMS**: Clients **access** the database through a **server**.
- **Distributed DBMS**: Data is distributed across **multiple servers**.
- **Cloud DBMS**: Database services are provided over the **cloud**.
- **Hybrid DBMS**: **Combines** features of centralized and distributed systems.
- **Peer-to-Peer DBMS**: Each node can act as a **client** and **server**.
- **In memory DBMS**: Data is stored in **RAM** for faster access.



Types of DBMS Architecture

There are several types of **DBMS architectures**:

- **Centralized DBMS**: All components are on a **single server**.
- **Client-Server DBMS**: Clients **access** the database through a **server**.
- **Distributed DBMS**: Data is distributed across **multiple servers**.
- **Cloud DBMS**: Database services are provided over the **cloud**.
- **Hybrid DBMS**: **Combines** features of centralized and distributed systems.
- **Peer-to-Peer DBMS**: Each node can act as a **client** and **server**.
- **In memory DBMS**: Data is stored in **RAM** for faster access.

Cadé
↳ Redis



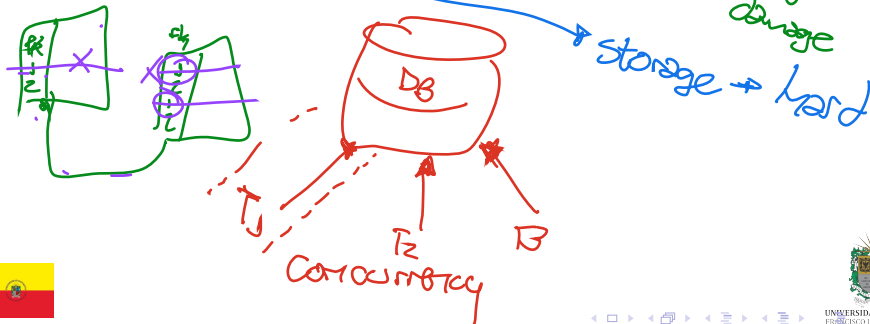
Outline

- 1 Database System Administration
- 2 Record Storage
- 3 DBMS Architecture
- 4 Transactional System**
- 5 Query Execution
- 6 Concurrency Control
- 7 Failure Recovery



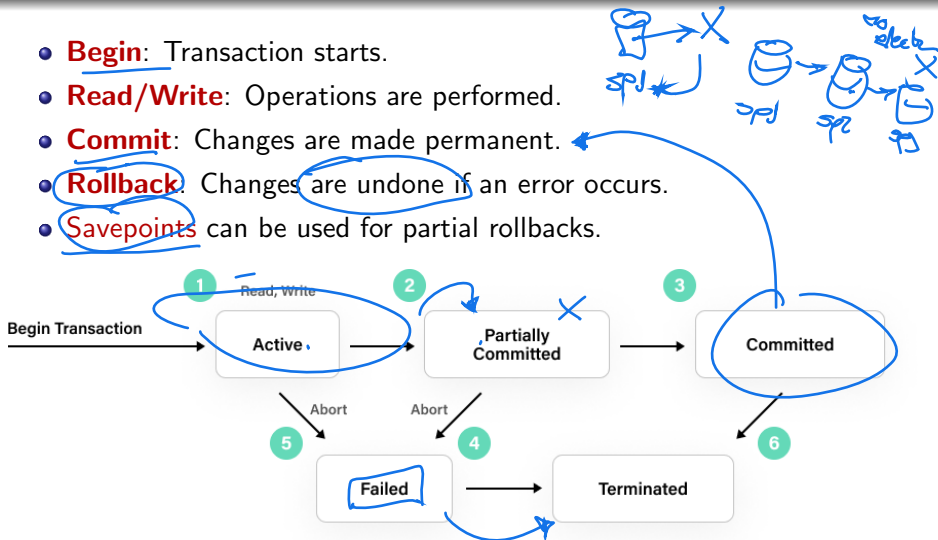
Transactional System Concepts

- A **transaction** is a sequence of operations performed as a **single** logical unit of work. *consulta → query*
- Transactions must satisfy the **ACID** properties:
 - **Atomicity**: All or nothing. *execution*
 - **Consistency**: Preserves database integrity.
 - **Isolation**: Transactions do not interfere.
 - **Durability**: Results persist after completion.



Transaction Lifecycle

- **Begin**: Transaction starts.
- **Read/Write**: Operations are performed.
- **Commit**: Changes are made permanent.
- **Rollback**: Changes are undone if an error occurs.
- **Savepoints** can be used for partial rollbacks.



Outline

- 1 Database System Administration
- 2 Record Storage
- 3 DBMS Architecture
- 4 Transactional System
- 5 Query Execution**
- 6 Concurrency Control
- 7 Failure Recovery



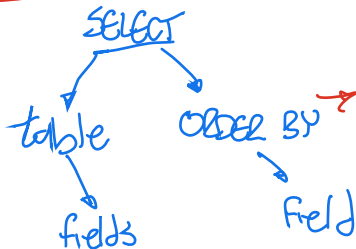
Query Execution Process

- **Query execution** is the process of interpreting and running database queries.

Steps:

- Parsing: Analyzing query syntax.
- Optimization: Choosing the best execution plan.
- Execution: Retrieving and processing data.

- Efficient execution is critical for performance.



~~INTO~~ ~~INSERT~~
INSERT INTO

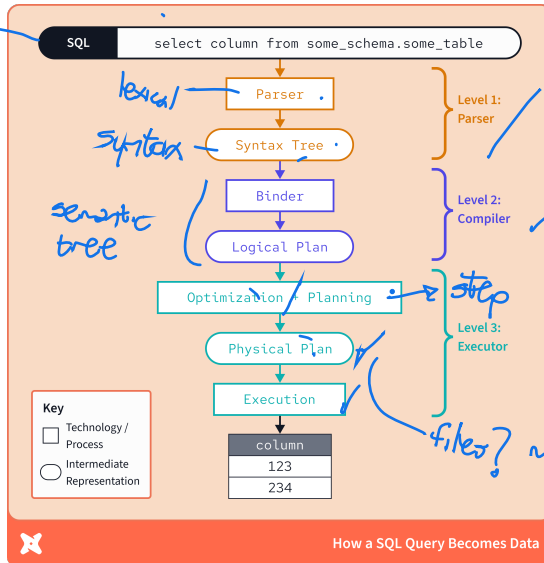
how? → engine

for (_)
if (_)

words
lexical
syntax → order



Query Execution Flow: Full Transaction



How a SQL Query Becomes Data



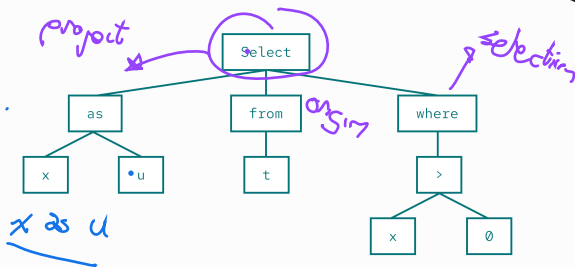
Query Execution Flow: Syntax Tree

SQL

select x as u from t where x > 0

Parser

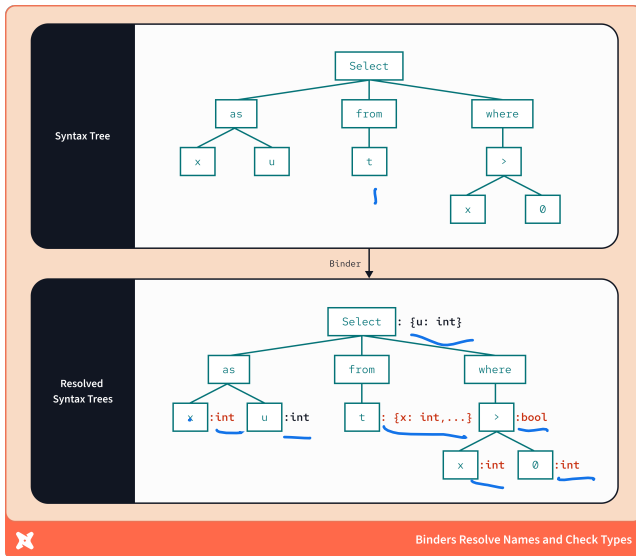
Syntax Tree



Parsers Recognize the Structure of the Query

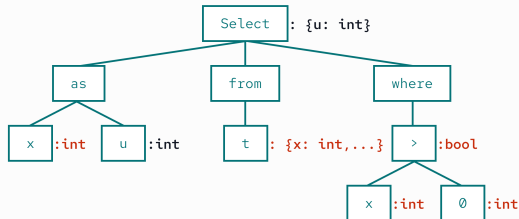


Query Execution Flow: Compilation



Query Execution Flow: Logical Plan

Resolved
Syntax Trees



Generating Logical Plan

Logical Plan

1. TableScan: t *→ for*
2. Filter: t.x > 0 *→ condn filter*
3. Projection: t.x as u *→ for x cond*



Compilation Produces an Executable Plan from a Resolved Syntax Tree



Query Optimization

- The query optimizer selects the most efficient strategy for executing a query.
 - Considers indexes, join methods, and data distribution.
 - May *re-write* queries for better performance.
 - Cost-based and rule-based optimization approaches.
- Handwritten notes:*
- memory
 - processor
 - amount & order
- Handwritten diagram:* A blue line under "strategy" with an arrow pointing down to the word "Plan" written in blue.



Query Optimization

- The **query optimizer** selects the most efficient strategy for executing a query.
- Considers **indexes**, **join methods**, and **data distribution**.
- May **rewrite queries** for better performance.
- **Cost-based** and **rule-based** optimization approaches.

column
 A J=10
 B X 11-15
 C 16-20
 ...

data



Query Optimization

- The **query optimizer** selects the most efficient strategy for executing a query.
- Considers **indexes**, **join methods**, and **data distribution**.
- May **rewrite queries** for better performance.
- **Cost-based** and **rule-based** optimization approaches.

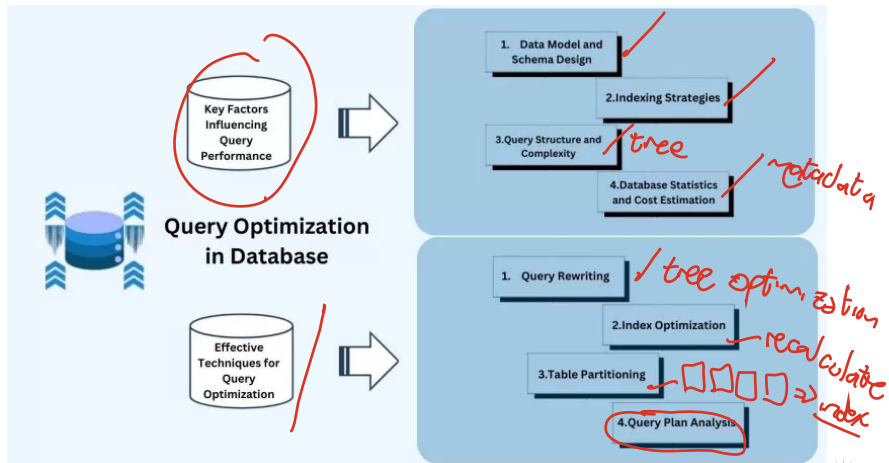


Query Optimization

- The **query optimizer** selects the most efficient strategy for executing a query.
- Considers **indexes**, **join methods**, and **data distribution**.
- May **rewrite queries** for better performance.
- **Cost-based** and **rule-based** optimization approaches.



Query Optimization Factors



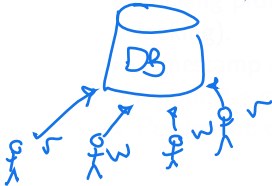
Outline

- 1 Database System Administration
- 2 Record Storage
- 3 DBMS Architecture
- 4 Transactional System
- 5 Query Execution
- 6 Concurrency Control**
- 7 Failure Recovery



Concurrency Control Concepts

- Concurrency control ensures correct results when multiple transactions execute simultaneously.
- Prevents problems like lost updates, dirty reads, and deadlocks.
- Techniques:



Concurrency Control Concepts

- **Concurrency control** ensures correct results when **multiple transactions** execute **simultaneously**.
- Prevents problems like **lost updates**, **dirty reads**, and **deadlocks**.
- Techniques:
 - **Locking protocols**: Use locks to control access (e.g., two-phase locking).

- **Timestamp ordering**: Assigns timestamps to transactions.
- **Optimistic concurrency control**: Assumes no conflicts.
- **Pessimistic concurrency control**: Assumes conflicts may occur.

Consistency + Integrity

Write → lock read
 read → lock write

- waiting time
 increase
 - memory due
 queues



Concurrency Control Concepts

- **Concurrency control** ensures correct results when **multiple transactions** execute **simultaneously**.
- Prevents problems like **lost updates**, **dirty reads**, and **deadlocks**.
- Techniques:
 - **Locking protocols**: Use locks to control access (e.g., two-phase locking).
 - **Timestamp ordering**: Assigns timestamps to transactions.

↳ T₁: J:J.00J
 T₂: J:J.00J
 Dirty Reads
 Fast



Concurrency Control Concepts

- **Concurrency control** ensures correct results when **multiple transactions** execute **simultaneously**.
- Prevents problems like **lost updates**, **dirty reads**, and **deadlocks**.
- Techniques:
 - **Locking protocols**: Use locks to control access (e.g., two-phase locking).
 - **Timestamp ordering**: Assigns timestamps to transactions.
 - **Optimistic concurrency control**: Assumes no conflicts **OCC**
 - **Pessimistic concurrency control**: Assumes conflicts may occur.

→ Parallel without locks
 ↳ reduce queues
 ↳ memory

high response time



Concurrency Control Concepts

- **Concurrency control** ensures correct results when **multiple transactions** execute **simultaneously**.
 - Prevents problems like **lost updates**, **dirty reads**, and **deadlocks**.
 - Techniques:
 - **Locking protocols**: Use locks to control access (e.g., two-phase locking).
 - **Timestamp ordering**: Assigns timestamps to transactions.
 - **Optimistic concurrency control**: Assumes no conflicts.
 - **Pessimistic concurrency control**: Assumes conflicts may occur.
- locking + timestamp



Locking and Deadlocks

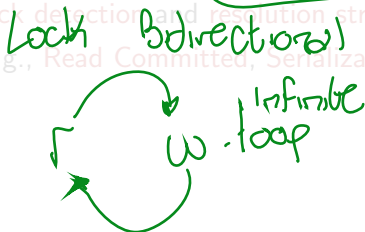
W. → 85
r. → 90

- **Locks** prevent conflicting operations on the same data.
- Deadlocks occur when transactions wait indefinitely for each other.
- DBMS uses deadlock detection and resolution strategies.
- Isolation levels (e.g., Read Committed, Serializable) control visibility of changes.



Locking and Deadlocks

- **Locks** prevent **conflicting operations** on the same data.
- **Deadlocks** occur when transactions **wait indefinitely** for each other.
- DBMS uses **deadlock detection and resolution strategies**.
- **Isolation levels** (e.g., Read Committed, Serializable) control visibility of changes.



Locking and Deadlocks

- **Locks** prevent **conflicting operations** on the same data.
- **Deadlocks** occur when transactions **wait indefinitely** for each other.
- DBMS uses **deadlock detection** and **resolution strategies**.
- Isolation levels (e.g., Read Committed, Serializable) control visibility of changes.

↓
loops?



Locking and Deadlocks

- **Locks** prevent **conflicting operations** on the same data.
- **Deadlocks** occur when transactions **wait indefinitely** for each other.
- DBMS uses **deadlock detection** and **resolution strategies**.
- **Isolation levels** (e.g., **Read Committed, Serializable**) control **visibility** of changes.

ACID



Outline

- 1 Database System Administration
- 2 Record Storage
- 3 DBMS Architecture
- 4 Transactional System
- 5 Query Execution
- 6 Concurrency Control
- 7 Failure Recovery**



Failure Recovery Concepts

- **Failure recovery** restores the database to a **consistent state** after a failure.

- Types of failures:

- Transaction failures: one transaction fails.
- System failures: all transactions are lost.
- Media failures: storage device fails.



- Recovery techniques:

- Write-ahead logging (WAL)
- Last stable version
- Metadata

last → good health



Failure Recovery Concepts

- **Failure recovery** restores the database to a **consistent state** after a failure.

- **Types of failures:**

- **Transaction failure:** Only one transaction fails.
- **System crash:** All active transactions are lost.
- **Media failure:** Disk or storage device fails.

- **Recovery techniques:**

- Write-ahead logging (WAL):

- Checkpointing:

domino effect
→ process close
↓
ACID



Failure Recovery Concepts

- **Failure recovery** restores the database to a **consistent state** after a failure.

- Types of failures:

- **Transaction failure**: Only one transaction fails.
- **System crash**: All active transactions are lost.
- **Media failure**: Disk or storage device fails.

- Recovery techniques:

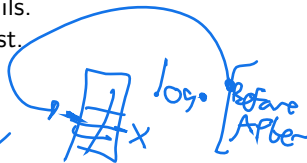
- **Write-ahead logging (WAL)**:
 - Logs changes before applying them.
 - Ensures durability and atomicity.

- **Checkpointing**:

- Periodically saves the database state.
- Reduces recovery time.

- **Shadow paging**:

- Maintains copies of data pages.
- Allows quick recovery to a consistent state.



Failure Recovery Concepts

- **Failure recovery** restores the database to a **consistent state** after a failure.
- Types of failures:
 - **Transaction failure**: Only one transaction fails.
 - **System crash**: All active transactions are lost.
 - **Media failure**: Disk or storage device fails.
- Recovery techniques:
 - **Write-ahead logging (WAL)**:
 - Logs changes before applying them.
 - Ensures durability and atomicity.
 - **Checkpointing**: *→ Backup* *→ Inconsistency*
 - Periodically saves the database state.
 - Reduces recovery time.
 - **Shadow paging**:
 - Maintains copies of data pages.
 - Allows quick recovery to a consistent state.



Failure Recovery Concepts

- **Failure recovery** restores the database to a **consistent state** after a failure.
- Types of failures:
 - **Transaction failure**: Only one transaction fails.
 - **System crash**: All active transactions are lost.
 - **Media failure**: Disk or storage device fails.
- Recovery techniques:
 - **Write-ahead logging (WAL)**:
 - Logs changes before applying them.
 - Ensures durability and atomicity.
 - **Checkpointing**:
 - Periodically saves the database state.
 - Reduces recovery time.
 - **Shadow paging**:
 - Maintains copies of data pages.
 - Allows quick recovery to a consistent state.



Write-Ahead Logging and Checkpoints

- **Write-ahead logging (WAL):** All changes are **logged before** being applied to the database.
- **Checkpoints:** Periodically **save** the database **state** to speed up recovery.
- **Shadow paging:** Maintains **copies** of data **pages**.
- **Recovery** ensures atomicity and durability.



Write-Ahead Logging and Checkpoints

- **Write-ahead logging (WAL):** All changes are **logged before** being applied to the database.
- **Checkpoints:** Periodically **save** the database **state** to speed up recovery.
- **Shadow paging:** Maintains **copies** of data **pages**.
- **Recovery** ensures atomicity and durability.



Write-Ahead Logging and Checkpoints

- **Write-ahead logging (WAL):** All changes are **logged before** being applied to the database.
- **Checkpoints:** Periodically **save** the database **state** to speed up recovery.
- **Shadow paging:** Maintains **copies** of data **pages**.
- Recovery ensures atomicity and durability.



Write-Ahead Logging and Checkpoints

- **Write-ahead logging (WAL):** All changes are **logged before** being applied to the database.
- **Checkpoints:** Periodically **save** the database **state** to speed up recovery.
- **Shadow paging:** Maintains **copies** of data **pages**.
- **Recovery ensures atomicity and durability.**



Outline

- 1 Database System Administration
- 2 Record Storage
- 3 DBMS Architecture
- 4 Transactional System
- 5 Query Execution
- 6 Concurrency Control
- 7 Failure Recovery



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/databases-ii>

